

Purpose:

The purpose of this report is to illustrate the divide between theoretical robotics and practical robotics and how they shouldn't be exclusive. In the industry the theoreticians often lack the perspective of physical reality thus even though what the mathematical model entails is superior, it is not easy to adapt with the hardware at hand. And on the other hand the practitioners of robotics are often unaware of what the math could do for them and are instead are stuck with the math that worked on hardware that was the peak of technology 30 years ago. This divide is caused by 2 main, 1 is the fallacy of assuming a lot of the standard functions we use being solved, and the fact that the industry lacks a unified way to talk about robotics as a whole, instead borrowing and merging different concepts from different fields of math which causes friction.

The Intuition

All of this came about from a question I had about transcendental functions, how does a computer solve for transcendental functions efficiently. In calculus one of the most powerful ways to express any continuous function is the functions Taylor Approximation which is essentially an infinite polynomial that behaves like the function centered around a point. However by definition a Taylor Series is infinite in size, and the problem with infinity is that it requires infinite time to compute. That is why we usually use a finite section of this approximation and accept that there is some inherent error. The industries solution for this for things like sin and cos, which are the core building blocks of robotics is a range reduction followed by a minimax polynomial which is a polynomial formulated in a way that minimizes the error we get from using a finite Taylor Approximation. However this method uses 2 processes that are inherently hard to do on a lot of hardware, one is the range reduction done by a modulo operator which gives you the remainder of when something is divided by a number and a lookup tables used to solve the minimax polynomials. That is what got me thinking about an alternative way that bypasses some of these hardware limitations.

How to leverage what we can from our functions?

Taylor Approximations are most accurate around where they are centered, this means for very small input numbers they are much more accurate, even for low order approximations. This is where things like the small angle approximations for trigonometric identities come from. So this gives us a cool insight on what we could do, if we work with small enough numbers trigonometric identities disappear, which is invaluable for ease of computation, but the problem is how we get there, and this is where binary comes in to save the day. A computer's natural language is binary and as such it stores the numbers in a way like this:

$$N = (-1)^n * Mantissa * 2^{exponent}$$

This means that the easiest way for a computer to scale a number down by a lot is by changing the exponent. This is referred to as bit shifting in computers. But this leaves us with the problem of scaling back up to get our actual function we want to evaluate, thankfully most trigonometric identities can be expressed in terms of smaller angles, for example for sin and cos the expression is:

$$\sin(a+b) = \sin(a) * \cos(b) + \cos(a) * \sin(b)$$

$$\cos(a+b) = \cos(a) * \cos(b) - \sin(a) * \sin(b)$$

This property becomes very powerful wen we realize there is a specialized version of this identity called the half trigonometric identity which for sin and cos can be expressed as:

$$\sin(2a) = 2 * \sin(a) * \cos(a) \quad \text{thus} \quad \sin(a) = 2 * \sin\left(\frac{a}{2}\right) * \cos\left(\frac{a}{2}\right)$$

$$\cos(2a) = \cos^2(a) - \sin^2(a) \quad \text{thus} \quad \cos(a) = \cos^2\left(\frac{a}{2}\right) - \sin^2\left(\frac{a}{2}\right)$$

This means we have a means of expressing trigonometric functions with bigger angles as inputs in terms of trigonometric functions with smaller angles as inputs. For sin and cos this process can be expressed like this

$$\theta_0 = \frac{\theta}{2^k}, \sin(\theta_0) \approx \theta_0, \cos(\theta_0) \approx 1, \sin(2\theta_0) = 2 * \sin(\theta_0) * \cos(\theta_0) = 2 * \theta_0 * 1, \cos(2\theta_0) = \cos^2(\theta_0) - \sin^2(\theta_0) = 1 - \theta_0^2$$

$$\sin(2 * 2\theta_0) = 2 * \sin(2\theta_0) * \cos(2\theta_0) = 2 * (2 * \theta_0) * (1 - \theta_0^2) \dots$$

A more general expression for this process can be notated like this:

$$S_0 = \theta_0, C_0 = 1, S_{i+1} = 2 * S_i * C_i, C_{i+1} = C_i^2 - S_i^2 \text{ for } 0 \leq i \leq k-1$$

This sort of iterative loop is a very computationally efficient way to express sin and cos, however as can be seen we can not isolate just sin and cos in this formulation, we need to compute both simultaneously, which shouldn't be all that surprising since they are essentially linked together in Euler's Formula which is $e^{i\theta} = \cos(\theta) + i * \sin(\theta)$. Which brings

up an interesting avenue, exponential function also have half exponent identities where $e^{\frac{\theta}{2}} = (e^{\frac{\theta}{2}})^2$. This means we can apply our bit shifting and scaling approach to exponential as well:

$$e^x = (e^{\frac{x}{2^k}})^{2^k}, \frac{x}{2^k} = x_0, X_0 = e^{x_0}, X_{i+1} = X_i^2 \text{ where } 0 \leq i \leq k-1, X_k = e^x$$

When applied to complex exponential this can be expressed as:

$$e^{ix} = e^{\frac{ix}{2^k}} = \cos(x_0) + i * \sin(x_0), \cos(x_0) \approx 1 - \theta_0^2 * 0.5, \sin(x_0) \approx x_0 \Rightarrow X_0 = 1 - \theta_0^2 * 0.5 + i * x_0, X_{i+1} = X_i^2 \text{ where } 0 \leq i \leq k-1, X_k = e^{ix}$$

This concept lays out the concept for what I try to do. Exponential functions are powerful, in the imaginary plane they represent rotations and now we have a way to express them in terms where we bypass the need to call trigonometric functions. This was the crux where I realized why stop here. Why stop at just exponentiating imaginary numbers, I know certain fields of math and physics take matrices as exponents, in most cases you would just use a Taylor Series Expansion, However there are a special subset of matrix exponentials where matrix A squared is -1, this is referred to as a skew symmetric matrix and this feels awfully similar to how the imaginary number "i" behaves and thus the Taylor Series just dissolves into $e^{A\theta} = I * \cos(\theta) + A * \sin(\theta)$. This is one of the core principles used in Lie Algebra and screw theory in robotics where a Rotation around an axis can be represented as a matrix exponential, just like how a complex exponential is a rotation in 2D, this represents a rotation in 3D. But again why stop here. What if we were to take vectors as exponents. This is where I had reached a roadblock because I didn't know what the multiplication of two vectors would be. The answer turns out an entirely different field of mathematics called Clifford Algebra or Geometric Algebra. How geometric algebra defines a geometric product is discussed with much more detail further along this report, in sum. the geometric product in 3D is essentially the dot product + i times the cross product. If we consider $e^{i * \vec{u} * \theta}$ due to the fact that u is a unit vector and its geometric product by itself is equal to 1 and i^2 is -1 we can express the exponential as $\cos(\theta) + i * \vec{u} * \sin(\theta)$ which is just a unit quaternion, but more importantly for our cause $\left(e^{\frac{i * \vec{u} * \theta}{2}}\right)^2 = e^{i * \vec{u} * \theta}$ thus we can use the same bit shifting logic then squaring method to get rid of the transcendental equations.

$$\frac{\theta}{2^k} = \theta_0, e^{i * \vec{u} * \theta_0} = \cos(\theta_0) + i * \vec{u} * \sin(\theta_0) \approx 1 - \theta_0^2 * 0.5 + i * \vec{u} * \theta_0 = X_0, X_{i+1} = X_i^2 \text{ for } 0 \leq i \leq k-1$$

At this point I was decently satisfied with what I had come up so I started looking for other stuff I could improve, and then I came across a concept called "Automatic Differentiation" or AD. It uses the imaginary number epsilon where $\epsilon^2 = 0$ but $\epsilon \neq 0$. It is used like this:

$$f(t + \epsilon) = \sum \frac{f^{(n)}(t)}{n!} * \epsilon^n = f(t) + f'(t) * \epsilon + \frac{f''(t)}{2} * \epsilon^2 \dots = f(t) + f'(t) * \epsilon \text{ thus } f'(t) = \text{Im}(f(t + \epsilon))$$

So we are able to find what the derivative of something without having to take the derivative. However this fact resonated with me, if ϵ essentially allows us to take the derivative, it linearized the curve at that point in the imaginary frame. So what would happen if I put $+\epsilon$ in a quaternion. This is how I got my introduction to geometric algebra.

Geometric Algebra

To answer the question of what the product of two vectors we need to define what a vector is and how it is different than a scalar. A scalar has a magnitude but doesn't have a direction whereas a vector has both direction and magnitude. In a matter of speaking they have different dimensions. So a more general question could be what is the sum of a scalar and vector. The general answer is nothing since they live on separate worlds. However this hasn't stopped us for imaginary and real numbers which also live on different worlds, so why should it stop us now. This is the essence of what geometric algebra is. It deals with the interactions of multiple dimensional operations simultaneously.

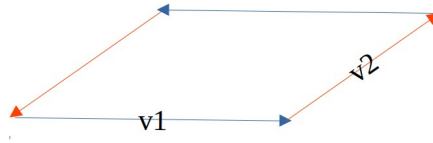
So lets take a simpler example. A 2D example where we have our scalar s and our vector $\vec{v}$ and lets say $z = s + \vec{v}$ by itself this doesn't mean much because we haven't decided what this expression actually means now lets consider what something like a sum would be which is decently straight forward.

$$z_1 = s_1 + \vec{v}_1, \quad z_2 = s_2 + \vec{v}_2, \quad z_1 + z_2 = s_1 + s_2 + \vec{v}_1 + \vec{v}_2$$

Now lets consider what a multiplication is for this z :

$$z_1 * z_2 = (s_1 + \vec{v}_1)(s_2 + \vec{v}_2) = s_1 + s_2 + s_1 * \vec{v}_2 + s_2 * \vec{v}_1 + \vec{v}_1 \vec{v}_2$$

We have reached the point at which we need to define what the product of 2 vectors are. In vector algebra we use 2 operators to represent something resembling as multiplication, these are the cross product and the dot product. We use the cross product when we want to represent how much something twists something, intuitively this could be represented using a cyclical sum of v_1 and v_2 , essentially form a parallelogram using v_1 and v_2 in a manner shown below:



In geometric algebra this is what we refer to as a bivector. It is cyclical by definition so it has a direction, and its magnitude can be expressed as the area of the parallelogram. However as you can see we have ran into a problem because in 2D the cross product isn't defined conventionally, it is indirectly defined by us looking through the z axis of a Cartesian frame and the cross product is pointing towards or away from us. However with how we showed the bivector as a cyclical sum we actually achieved something better than the cross product because we were able to represent it in 2D or any higher dimension as well. This operation in geometric algebra is referred to as the wedge product or the outer product and it is the measure of twist. However technically speaking this approach doesn't represent the full scope of multiplication because there is no area to speak of when both vectors are aligned, regardless of the magnitude of the vectors involved, and that is why we use the outer product in conjunction with the inner product, which we commonly refer to as the dot product. This combination is what we refer to as the geometric product, and can be expressed as such:

$$\vec{v}_1 \vec{v}_2 = \vec{v}_1 \cdot \vec{v}_2 + \vec{v}_1 \wedge \vec{v}_2$$

This means the geometric product of two vectors in 2D yield a scalar and a bivector. Bivectors in 2D behave similar to scalars because even though they have directions and magnitudes, the direction is essentially mimicking what the sign of a scalar does, this is why in 2D we refer to them as pseudo scalars. And with this we can see that the 2D world deals with a scalar, a vector, and a bivector. Now lets consider the physical significance of a bivector. As we established the bivector is the measure of twist.

$$\vec{v}_1 = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \vec{v}_2 = \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}, \vec{\vec{V}} = \vec{v}_1 \wedge \vec{v}_2, \quad \vec{v}_1 \vec{\vec{V}} = \vec{v}_1 \wedge \vec{v}_1 \wedge \vec{v}_2 + \vec{v}_1 \cdot \vec{v}_1 \wedge \vec{v}_2 = \vec{v}_2$$

$$\vec{\vec{V}} \vec{v}_1 = \vec{v}_1 \wedge \vec{v}_2 \cdot \vec{v}_1 + \vec{v}_1 \wedge \vec{v}_1 \wedge \vec{v}_2 = -\vec{v}_2$$

From this we can think of a bivector in 2D as a rotation. But what if we define a bivector in the form like this:

$$\vec{v}_1 = e_1, \vec{v}_2 = e_2, \vec{\vec{V}} = e_1 \wedge e_2 = e_{12}, \vec{v}_3 = a * e_1 + b * e_2, \quad \vec{\vec{V}} \vec{\vec{V}}$$

$$\vec{\vec{V}} \vec{v}_3 = e_1 \wedge e_2 (a * e_1 + b * e_2) = a * (e_1 \wedge e_2 \cdot e_1) + b * (e_1 \wedge e_2 \cdot e_2) = -a * e_2 + b * e_1 = \begin{bmatrix} b \\ -a \end{bmatrix}$$

$$\vec{\vec{V}} (b * e_1 - a * e_2) = e_1 \wedge e_2 \cdot (b * e_1 - a * e_2) = b * (e_1 \wedge e_2 \cdot e_1) - a * (e_1 \wedge e_2 \cdot e_2) = -a * e_1 - b * e_2 = \begin{bmatrix} -a \\ -b \end{bmatrix}$$

From this we can see that in 2 dimensions this bi vector squared is equal to -1 and by itself it rotates vectors by 90 degrees. Bivectors in 2D that are defined as the wedge of 2 perpendicular unit vectors is defined as “I”. This is the exact i that we define as the imaginary number that is the square root of -1. and as such the complex exponential is essentially a tool that rotates vectors around the origin.

In 3D we deal with a similar setup but with a couple of differences. Firstly we have another n-vector, this time the trivector which is the outer product of a vector and bivector and its magnitude represents volume and direction represents “handedness”. Also in 3D the trivector is referred to as the pseudo scalar and the bivector is referred to as the pseudo vector due to how they behave.

$$\vec{v} = a * e_1 + b * e_2 + c * e_3, \vec{\vec{V}} = k * e_{12} + l * e_{13} + m * e_{23}, \vec{\vec{\vec{W}}} = n * e_{123} = n * I$$

$$\vec{\vec{V}} \wedge \vec{v} = (k * e_{12} + l * e_{13} + m * e_{23}) \wedge (a * e_1 + b * e_2 + c * e_3) = k * e_{123} - l * e_{123} + m * e_{123} = e_{123} * (k - l + m)$$

$$e_{12} = i e_3 = e_1 e_2 e_3 e_3 = e_1 e_2 = e_{12}, e_{13} = -i e_2 = -e_1 e_2 e_3 e_2 = e_1 e_2 e_3 = e_1 e_3 = e_{13}, e_{23} = -i e_1 = -e_1 e_2 e_3 e_1 = e_1 e_2 e_3 = e_{23}$$

$$\vec{\vec{V}}^2 = -1, \vec{\vec{V}}^{-1} \vec{\vec{V}}^2 = -\vec{\vec{V}}^{-1} = \vec{\vec{V}}, \vec{\vec{V}}^{-1} = -\vec{\vec{V}}$$

$$\vec{\vec{V}} = i * (\vec{u}) \text{ where } \vec{u} = e_3 \text{ thus } \vec{\vec{V}} = e_{12}, \vec{v} = a * e_1 + b * e_2 + c * e_3, \vec{\vec{V}} \vec{v} = e_{12} (a * e_1 + b * e_2 + c * e_3) = -a * e_2 + b * e_1 + c * e_{123}$$

$$\vec{\vec{V}}^{-1} \vec{v} = -b * e_1 + a * e_2 - c * e_{123}$$

$$\vec{\vec{V}}^{-1} \vec{\vec{V}} \vec{v} = -e_2 e_1 (a * e_1 + b * e_2 + c * e_3) e_1 e_2 = a (-e_2 e_1 e_1 e_1 e_2) + b (-e_2 e_1 e_2 e_1 e_2) + c (-e_2 e_1 e_3 e_1 e_2) = -a * e_1 - b * e_2 - c * e_3$$

From these expressions we can see that when a unit bivector is multiplied by a vector, it rotates it along a rotation axis by +90 degrees, and when the inverse is multiplied by a vector, it is rotated -90 degrees along the rotation axis. But when sandwiched around the vector the resultant rotation is 180 degrees. Due to how a single bivector rotates by 90 degrees

we can express this like $e^{i * \vec{u} * \frac{\pi}{2}} = \cos(\frac{\pi}{2}) + i * \vec{u} * \sin(\frac{\pi}{2}) = i * \vec{u} = e_{12}$. So to get a theta degrees worth of rotation along an

axis we use $e^{-i * \vec{u} * \frac{\theta}{2}} * \vec{v} * e^{i * \vec{u} * \frac{\theta}{2}}$ where $e^{i * \vec{u} * \frac{\theta}{2}} = \cos(\frac{\theta}{2}) + i * \vec{u} * \sin(\frac{\theta}{2})$. This is what we define as a unit quaternion and in geometric algebra they are referred to as rotors because they rotate vectors.

Projective Geometric Algebra

Projective Geometric Algebra or PGA is a sub branch of Geometric Algebra that has a slightly modified approach to Geometric algebra. In PGA vectors are planes, bivectors are lines, and trivectors are points. The fundamental difference between the two is that even though regular geometric algebra handles rotations very well it struggles with translations. PGA approaches by defining a new unit axis we’ll call e_0 which represents infinity. This is useful due to the fact that a translation can be expressed as just a rotation around an axis that is infinitely far away. e_0

is defined using the relations $e_0^2 = 0$ but $e_0 \neq 0$. In PGA our fundamental objects can be expressed as the following:

$$\vec{v} = a * e_1 + b * e_2 + c * e_3 + d * e_0, L = a * e_{01} + b * e_{02} + c * e_{03} + x * e_{12} + y * e_{13} + z * e_{23}, P = e_{123} + x * e_{032} + y * e_{013} + z * e_{021}$$

Motors

In these expressions the plane and points make intuitive sense but lines are a bit more obscure at first glance. But essentially lines are defined using Plücker Coordinates or can be thought of simply as the meet or the outer product of 2 planes. Bivectors in PGA, when their real part is a unit quaternion, their squares are equal to 1. By using this fact

that we can try exploiting the fact that $e^{L\theta} = \cos(\theta) + L * \sin(\theta)$ This would represent a rotor where there is a revolute joint where no translation takes place. However by making use of the properties of ϵ_0 . If we consider $\hat{\theta} = \theta + d * e_0$ then replaces the theta with theta hat in the formula we get

$$e^{L\hat{\theta}} = \cos(\hat{\theta}) + L * \sin(\hat{\theta}) = \cos(\theta) - \sin(\theta) * e_0 * d + L * (\sin(\theta) + e_0 * d * \cos(\theta)) \text{ where } L = \vec{u} + e_0 \vec{r} \wedge \vec{u}$$

$$e^{L\hat{\theta}} = \cos(\theta) - \sin(\theta) * e_0 * d + (\vec{u} + e_0 \vec{r} \wedge \vec{u}) * (\sin(\theta) + e_0 * d * \cos(\theta)) = \cos(\theta) + \vec{u} * \sin(\theta) + e_0 (\vec{u} * d * \cos(\theta)) + \vec{r} \wedge \vec{u} * \sin(\theta) - d * \sin(\theta)$$

However the same situation with regular quaternions exist with dual quaternions where we use this product in a sandwich and as a result to get the right scale of rotation and translation we need to work with half of theta hat. This dual quaternion math allows us to define the joints in a unified manner.

The geometric Jacobian

The Jacobian is the map that translates one frame to another. Normally for this we use partial derivatives where the 6 degrees of freedom are mapped onto all our input parameters, and using partial derivatives we measure how much they are affected by our input parameters. This process is decently straight forward but it is quite heavy computationally because we have to store a lot of data and take a lot of partial derivatives. PGA has a neat way of lessening our load for calculating jacobians because it doesn't need partial derivatives. Instead it can tap into the same concept of screw theory where along the axis of instantaneous zero velocity or the screw axis which we defined as L, the motion can simply be expressed as a rotation along said axis. This means that our bivector L is the column of the Jacobian, because by definition $V = J * \dot{q}$ and $L = \vec{u} + e_0 * \vec{r} \wedge \vec{u} = \vec{u} + e_0 \vec{u}'$ and $(\vec{u} + e_0 \vec{u}') * (\omega + e_0 * \dot{\theta}) = \vec{u} \omega + e_0 (\vec{u} * \dot{\theta} + \vec{u}' \omega)$ is just the angular

velocity and the linear velocity. To find L's we just use $(\prod_1^i M_i) L_i (\prod_i^0 \widetilde{M}_i)$ where L_i is the screw axis at the home

position, M_i is the motor with the indice i and $\widetilde{M}_i$ is the Motor with $-\hat{\theta}$. This also slightly gives us a more intuitive way to think about singularities as well because for us to lose a degree of freedom means that one of our screw axis is aligned with another. And these are faster to compute than evaluating a full determinant and it also gives us exactly which axes aligned meaning we can deal with these singularities in a more informed way.

IK and FK

The fundamental building blocks of robotics are the IK and FK. PGA is very well equipped to handle things like FK since by using dual quaternions we are able to represent both rotations and translation we can essentially construct anything a DH matrix can and even more, because by definition a dual quaternion has 6 degrees of freedom whereas a DH matrix has 4 degrees of freedom. This makes it a very powerful tool when it comes to FK. However it is also quite powerful when it comes to the IK of parallel robots due to how it handles objects in terms of intersections and regressions of other objects. For things like agile eyes this is incredibly powerful due to fact that we are able to solve for links with different arc lengths meaning we are not bound to the regular 90 degree fixed links found in traditional agile eyes. This gives us control over where the singularities are and how to better manage them. The process to define the IK of an agile eye can be seen below: